Introduction to Programming and Data Structures

Data Structures - Linked Lists

Malay Bhattacharyya

Associate Professor

MIU, CAIML, TIH
Indian Statistical Institute, Kolkata

October, 2023

Problems with arrays :
1) insertion involves shifting of each element, O(n)
2) when dynamic array capcity is full, reallocation increases cost and efficiency

1. Linked Lists
   - Basics

1) don't need to pre-allocate space
2) insertion is easier

2. Implementing a Linked List
   - Standard implementation

3. Problems

Access/Search/Insertion/Deletion at a Specific Position (O(n)): In the worst case, traverse the entire list to find an element

Insertion/Deletion at the Beginning (O(1)): Adding or removing an element at the beginning of the list takes constant time. You only need to update a few pointers.

Insertion/Deletion at the End (O(n)): If you don't keep a reference to the tail, adding or removing an element at the end of the list requires traversing the entire list , O(n).

Traversal (O(n)): Visiting all elements in the list requires traversing , O(n).

Motivating example

Problem: Maintain a database of student information for the class teacher: roll number, name, stream, courses taken, etc.
Data structure: Linked list (AIAO)
Operations:

- `INSERT`
  - at the beginning, end, other position (by index)
  - before / after a specific element
- `DELETE`
  - at the beginning, end, other position (by index)
  - before / after a specific element
- `LENGTH`: number of elements in the list
- `GET`: return the value of an item by index
- `SEARCH`: lookup
- `ITERATE` or `FOREACH`: to do something
- `SORT`: on a specified attribute

Standard implementation – Initialization

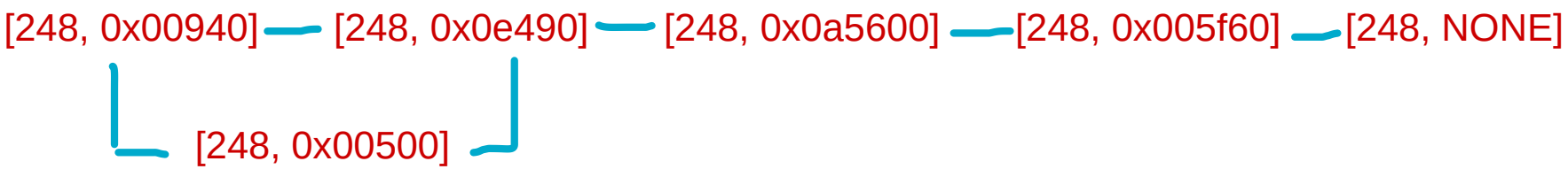
Considering the linked list size as static:

```
LinkedListDS = []
SizeLinkedListDS = 10
```

- it is a collection of elements, called nodes, where each node has two components:
- Data: This component stores the actual value or information that the node holds. It can be any data type, such as an integer, string, or a more complex data structure.
- Reference (or Pointer): This component points to the next node in the sequence. In a singly linked list, each node points to the following node. In a doubly linked list, each node has two references, one pointing to the next node and one pointing to the previous node.
- The nodes are connected in a linear sequence, which can be thought of as a chain of elements. The last node in the list typically has a reference that points to a special value (often None in Python) to indicate the end of the list.

Doubly linked list is a type of linked list in which each node has two references (or pointers): one pointing to the next node in the sequence, and another pointing to the previous node. This bidirectional navigation allows for more efficient traversal in both the forward and backward directions

Standard implementation – Insert operation



location/reference

```
def LinkedListINSERT(QueueDS, Data, Index):
    if len(LinkedListDS) < SizeLinkedListDS:
        LinkedListDS.insert(index, Data)
    else:
        print("Linked List overflow!!!")
```

Standard implementation – Delete operation

```
def LinkedListDELETE(LinkedListDS, Index):
    if len(QueueDS) > 0:
        Data = LinkedListDS.remove(Index)
        return Data
    else:
        print("Linked List underflow!!!")
```

Note: If you do not wish to receive the deleted element, exclude the return statement.

Standard implementation – Displaying the elements

```
def LinkedListDISPLAY(LinkedListDS):
    print("The elements in the linked list are: ")
    for Element in LinkedListDS:
        print(Element)
```

dictionary (key:value)
reference:data

Note: You can directly write `print(LinkedListDS)`.

Problems

1. Write a program in Python to find out the middle element in a linked list by having only one pass through the list. Consider that you are not aware of the length of the list and computing the length itself takes a single pass.
2. Write a program in Python to reverse a linked list.
3. Given two polynomials represented by two different linked lists, write a program in Python to add the polynomials. Note that adding the polynomials denotes adding the coefficients who have same variable powers represented within the linked lists.